QA Audit Report
Chat & Revenue Systems

Driftless (FocusBuddy) | March 17, 2026

Principal QA Auditor Assessment | FAANG-Level Review

Table of Contents

1. Executive Verdict

Chat Systems: PARTIALLY TESTED

The chat store (chatStore.ts) has 100% statement coverage with 32 unit tests covering all 5 actions (fetchMessages, sendMessage, clearHistory, deleteMessage, checkRateLimit). The service layer (chat.service.ts) has 7 tests covering CRUD operations with error paths. However: (a) the AI chat Edge Function (supabase/functions/ai-chat/index.ts) has ZERO automated tests, (b) the ChatScreen UI component (chat.tsx, 348 lines) has ZERO component/integration tests, (c) MessageBubble has ZERO render tests, and (d) timeout behavior (withTimeout wrapper) is not tested in the chat context.

Revenue Systems: PARTIALLY TESTED

The subscription store (subscriptionStore.ts) has 88% statement coverage with 20 tests across all 5 actions. The subscription service has 11 tests including trial status edge cases (expired, null trial_ends_at, within-24-hours). However: (a) the RevenueCat webhook handler (revenuecat-webhook/index.ts, 279 lines) has ZERO automated tests despite handling INITIAL_PURCHASE, RENEWAL, CANCELLATION, EXPIRATION, and BILLING_ISSUE events, (b) the activate-trial Edge Function has ZERO tests, (c) the delete-account Edge Function has ZERO tests, and (d) no integration or E2E tests exist for the full purchase flow.

2. Chat Functionality Audit

Feature	Test Files	Test Count	Rating	Gaps
Message fetch + pagination	chatStore.test.ts chat.service.test.ts	7 + 3 = 10	**DEEP**	No test for max pagination boundary
Message send (optimistic UI)	chatStore.test.ts	7	**DEEP**	Timeout behavior not tested; concurrent sends not tested
Message deduplication	chatStore.test.ts	1	**PARTIAL**	Only one dedup scenario tested
Message delete (optimistic + revert)	chatStore.test.ts chat.service.test.ts	3 + 2 = 5	**DEEP**	No test for deleting non-existent message
Clear history	chatStore.test.ts chat.service.test.ts	4 + 1 = 5	**DEEP**	None significant
Client-side rate limiting	chatStore.test.ts	4	**DEEP**	Boundary at exactly RATE_LIMIT_MAX-1 tested
Server-side rate limiting (ai-chat)	NONE	0	**NOT TESTED**	Edge Function has no automated tests; increment_rate_limit RPC untested
AI response generation (OpenAI call)	NONE	0	**NOT TESTED**	OpenAI integration, timeout (15s AbortController), error responses untested
Input sanitization (sanitizeForPrompt)	NONE	0	**NOT TESTED**	HTML removal, newline collapse, unicode filter untested
Shame language detection	NONE	0	**NOT TESTED**	20 shame patterns + integration with shame-emergency routing untested
Message persistence (DB insert)	NONE	0	**NOT TESTED**	Edge Function saves both user + assistant msgs; insert failure path untested
Auth enforcement	chatStore.test.ts chat.service.test.ts	3 + 0 = 3	**PARTIAL**	Client-side auth tested; server-side JWT validation in Edge Function untested
Cross-user deletion prevention (user_id filter)	chatStore.test.ts chat.service.test.ts	1 + 1 = 2	**DEEP**	Verified both .eq() filters are applied
ChatScreen UI (348 lines)	NONE	0	**NOT TESTED**	Optimistic UI, error display, rate limit banner, load-more, prefill param untested
MessageBubble component	NONE	0	**NOT TESTED**	Rendering, long-press delete, timestamp formatting, React.memo untested
Timezone validation (ai-chat)	NONE	0	**NOT TESTED**	slice(0,64) guard and Intl.DateTimeFormat fallback untested

Chat Totals: 39 tests across store + service layers. 0 tests for Edge Function, UI components, or integration flows.

3. Revenue Functionality Audit

Feature	Test Files	Test Count	Rating	Gaps
Subscription state derivation (deriveFrom CustomerInfo)	subscriptionStore.test.ts	4	**DEEP**	Active, free, trial, transient-error paths all tested
RevenueCat initialize	subscriptionStore.test.ts	2	**PARTIAL**	No-API-key + error-resilience tested. Happy path (configure+logIn+getCustomerInfo) not fully verified due to mock structure
Check entitlement	subscriptionStore.test.ts	4	**DEEP**	Pro, free, trial (with daysLeft), transient error resilience all tested
Fetch offerings	subscriptionStore.test.ts	3	**DEEP**	Current offering, null offering, error resilience tested
Purchase package	subscriptionStore.test.ts	4	**DEEP**	Success, isPurchasing flag, user cancellation, payment failure all tested
Restore purchases	subscriptionStore.test.ts	4	**DEEP**	Pro restore, free restore, isRestoring flag, error all tested
Subscription DB service (get/update/checkTrial)	subscription.service.test.ts	11	**DEEP**	Trial status edge cases (expired, null, within-24h, non-trialing status) comprehensively tested
Safe field whitelist (SEC-02)	subscription.service.test.ts	1 (implicit)	**PARTIAL**	updateSubscription tested but no explicit test proving dangerous fields (status, entitlement) are stripped
Trial activation (activate-trial Edge Function)	NONE	0	**NOT TESTED**	Atomic CAS guard, rate limiting (5/hr), already-subscribed (409), missing-row insert, auth all untested
RevenueCat webhook handler	NONE	0	**NOT TESTED**	HMAC-SHA256 verification, constant-time comparison, event mapping (5 types), idempotency (last_webhook_event_id), UUID validation, upsert/update paths all UNTESTED
Delete account (billing impact)	NONE	0	**NOT TESTED**	Soft-delete, rate limiting (3/hr), sign-out after deletion, auth all untested
Upgrade/downgrade flow	NONE	0	**NOT TESTED**	No code or tests for plan changes; handled entirely by RevenueCat SDK
Refund/reversal handling	NONE	0	**NOT TESTED**	No REFUND event type in webhook handler switch statement
Billing issue handling	NONE	0	**NOT TESTED**	BILLING_ISSUE event mapped in webhook but never tested; no grace period logic
Subscription UI (settings.tsx)	NONE	0	**NOT TESTED**	No component tests for subscription display, manage/restore buttons

Revenue Totals: 29 tests across store + service layers. 0 tests for any of the 3 Edge Functions (activate-trial, revenuecat-webhook, delete-account) or subscription UI.

4. Critical Revenue Risks (Top 5)

RISK-1: RevenueCat Webhook Has Zero Tests (CRITICAL)

The revenuecat-webhook Edge Function (279 lines) processes all subscription lifecycle events (INITIAL_PURCHASE, RENEWAL, CANCELLATION, EXPIRATION, BILLING_ISSUE). It contains HMAC-SHA256 signature verification with constant-time comparison, event-to-subscription mapping, idempotency checking via last_webhook_event_id, UUID validation, and upsert/update branching. NONE of this is tested. A regression in any of these paths could silently grant or revoke pro access, accept forged webhook events, or corrupt subscription records.

RISK-2: Trial Activation Edge Function Has Zero Tests (HIGH)

The activate-trial Edge Function contains atomic CAS guard logic (.not('status','in',...)) to prevent TOCTOU race conditions, rate limiting via increment_rate_limit RPC, and a fallback INSERT path for users without subscription rows. None of these paths are tested. A bug could allow unlimited free trials or block legitimate trial activations.

RISK-3: No REFUND Event Handling in Webhook (HIGH)

The webhook handler's switch statement covers INITIAL_PURCHASE, RENEWAL, CANCELLATION, EXPIRATION, and BILLING_ISSUE, but there is NO case for REFUND or PRODUCT_CHANGE events. RevenueCat sends these events. A refund would not revoke pro access, meaning users could get refunds while keeping paid features.

RISK-4: No Integration/E2E Test for Purchase Flow (HIGH)

The full purchase flow (user taps package -> RevenueCat SDK -> webhook -> DB update -> entitlement check) is tested only as isolated unit mocks. There are zero integration tests that verify the complete chain. If any layer's assumptions about data shapes change, the purchase flow could silently break.

RISK-5: Safe Field Whitelist Not Explicitly Tested (MEDIUM)

subscription.service.ts has a SafeSubscriptionUpdate type that should prevent clients from setting 'entitlement' or 'status' directly. While the TypeScript type system provides compile-time protection, there is no runtime test proving that a malicious payload with { status: 'active', entitlement: 'pro' } would be stripped. The test passes an 'any' cast.

5. Blind Spots / Missing Evidence

• **Edge Function Tests:** All 6 Supabase Edge Functions (ai-chat, ai-checkin, activate-trial, delete-account, data-export, revenuecat-webhook) have ZERO automated tests. These run in a Deno runtime and would require a separate test harness (e.g., Deno.test + fetch mocking). This is the single largest blind spot in the entire codebase.

• **Component/Integration Tests:** No React Native component tests exist for ChatScreen, MessageBubble, TypingIndicator, QuickActions (chat), or any settings/subscription UI. The Testing Library (@testing-library/jest-native) is installed but unused for screen-level tests.

• **E2E Tests:** Zero end-to-end tests exist. No Detox, Maestro, or Appium configuration found.

• **RLS Policy Tests:** Row-Level Security policies exist on all tables but are not tested via automated assertions. A misconfigured RLS policy could expose chat messages or subscription data across users.

• **Webhook Replay/Idempotency:** The webhook uses last_webhook_event_id for deduplication, but this has zero test coverage. A replay attack or RevenueCat retry could cause double-processing.

• **HMAC Signature Verification:** The constant-time comparison in verifySignature() is critical for preventing timing attacks. This cryptographic code has zero unit tests.

• **Concurrent Purchase Attempts:** No tests verify behavior when a user simultaneously triggers purchasePackage() from two devices or rapidly taps the purchase button.

• **Network Failure Recovery:** No tests for chat or subscription behavior during intermittent network failures, offline mode, or partial responses.

6. Final Production Risk Assessment

Overall Production Risk: MEDIUM-HIGH

Justification:

The client-side state management and service layers are well-tested (86.79% store coverage, 93.38% service coverage, 381 passing tests). However, all server-side business logic - the 6 Edge Functions that handle revenue-critical operations (webhook processing, trial activation, AI chat), security (HMAC verification, rate limiting), and data persistence - has ZERO automated test coverage. This creates a dangerous asymmetry: the code that users interact with is tested, but the code that handles money and security is not.

Layer	Test Count	Coverage	Risk Level
Zustand Stores (client state)	72 tests / 4 suites	86.79% stmt	LOW
Service Layer (DB operations)	~170 tests / 11 suites	93.38% stmt	LOW
Edge Functions (server logic)	0 tests / 0 suites	0%	CRITICAL
UI Components (screens)	~5 tests / 3 suites	<10%	MEDIUM
E2E / Integration	0 tests	0%	HIGH

Recommended Priority Actions (Ordered by Revenue Impact):

1. Write Deno integration tests for revenuecat-webhook (HMAC verification, all 5 event types, idempotency, error paths) - CRITICAL
2. Write Deno tests for activate-trial (CAS guard, rate limiting, already-subscribed, insert fallback) - HIGH
3. Add REFUND and PRODUCT_CHANGE event handling to webhook handler - HIGH
4. Write Deno tests for ai-chat (sanitization, rate limiting, timeout, OpenAI error handling) - HIGH
5. Add component tests for ChatScreen and subscription settings UI - MEDIUM
6. Add RLS policy integration tests (cross-user data isolation) - MEDIUM
7. Set up E2E test framework (Detox or Maestro) for critical user flows - MEDIUM
